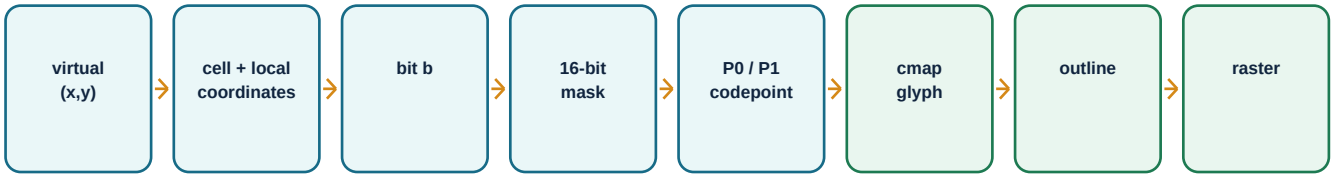
PUA 4x4

Evidentiary proof from a 4 x 4 virtual-pixel grid to the installed Linux glyph raster

CORRECTED CONCLUSION	**Font v0.3 implements the intended MSB-left mapping: inside each row the visible columns correspond to bits 3,2,1,0. Font v0.2 was internally self-consistent but used the opposite LSB-left convention. Font v0.1 also had unequal exterior outlines. v0.3 preserves v0.2's exact 125 x 250 geometry and changes the cmap/glyph construction so every codepoint now has the requested meaning.**



Each gate must pass before the next representation is trusted.

Scope and method

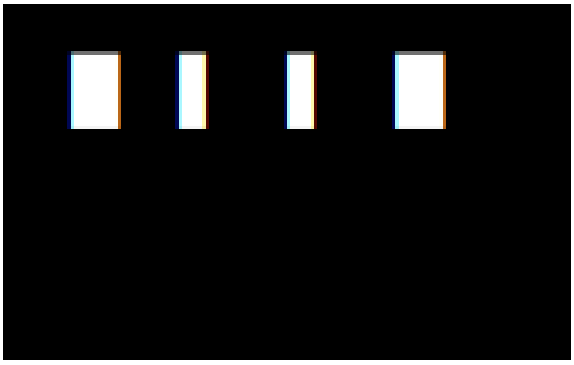
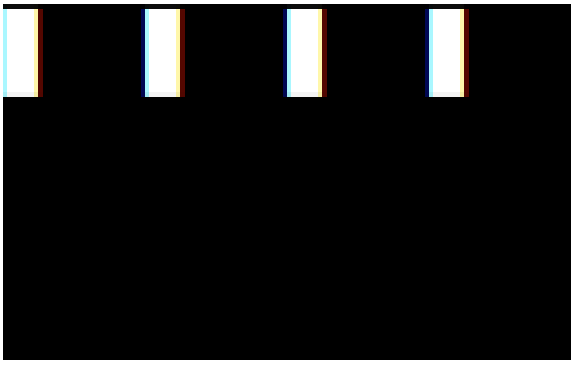
This v1.2 report supersedes v1.1. Earlier audits proved internal consistency, but they did not compare the chosen bit convention with the requested convention. The user's character-editor calculation supplied that missing semantic requirement: local x=0 must select the most-significant bit of its four-bit row, not the least-significant bit.

The corrected audit begins without trusting the demo's encoder. It recalculates every coordinate, bit, mask and codepoint independently, parses both TrueType binaries directly, verifies every composite component and its exact bounds, and then checks the installed Linux Fontconfig/Pango path. There are 65,536 possible 16-bit masks; all 65,536 are tested.

Each numbered gate below states the evidence required before the following gate can be accepted. Machine-readable results accompany this PDF.

0. Failed criterion discovered by the trail test

The trail test moved one virtual pixel left one position at a time. The selected masks and codepoints advanced correctly, yet the visible mark changed width. That observation isolates the failure after codepoint selection: the component outlines themselves were not equal.

v0.1 - defective edge overfill	v0.3 - equal geometry plus MSB-left mapping
	
Measured raster widths for local x=0,1,2,3: **16, 10, 10, 16 pixels**. All marks were 23 pixels high. Edge components extended outside the cell.	Measured raster widths for local x=0,1,2,3: **10, 10, 10, 10 pixels**. All marks are 16 pixels high. No component crosses the cell boundary.

Root cause

The nominal subcell is 125 x 250 font units. In v0.1, the generator added 100 units of overfill at each exterior edge. A left or right edge pixel therefore became 225 units wide, while an interior pixel remained 125. A top or bottom edge pixel became 350 units high, while an interior pixel remained 250. This was intended to hide raster seams, but it violated uniform subpixel addressing and caused the observed jumps.

CORRECTION GATE - equal isolated pixels

Criterion to advance: Every one-bit glyph must have identical 125 x 250 outline bounds translated only by its local (x,y), and every bound must stay within x=0..500 and y=-200..800.

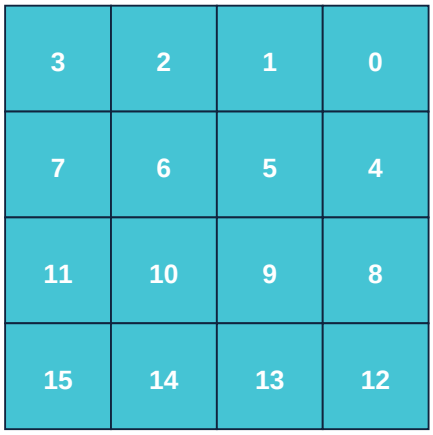
Measured evidence: The v0.3 binaries were parsed directly. All sixteen reusable components in both fonts have exactly 125 x 250 bounds and no component leaves the character cell.

Result: PASS. The physical geometry remains exact while the mapping convention changes.

1. Establish the coordinate systems

A terminal has character cells. PUA 4x4 treats each character cell as sixteen addressable virtual pixels. Virtual coordinates and cell coordinates are zero-based. ANSI cursor positions are one-based.

one terminal cell: local bits 0..15




```
For a virtual pixel (x, y):

character_column = x div 4
character_row = y div 4
local_x = x mod 4
local_y = y mod 4
bit b = 4 x local_y + (3 - local_x)

ANSI column = character_column + 1
ANSI row = character_row + 1
```

Worked coordinate example

Virtual pixel (13, 10) maps to terminal cell (3, 2), local position (1, 2), bit b = 4 x 2 + (3 - 1) = 10, bit value 1 << 10 = 0x0400, and ANSI cursor position row 3, column 4.

GATE 1 - coordinate arithmetic

Criterion to advance: All sixteen local positions must map bijectively to bits 0..15 in the requested orientation.

Measured evidence: The independent audit enumerated local_y=0..3 and local_x=0..3 using b=4*local_y+(3-local_x). The resulting set is exactly {0..15}; each row reads 3,2,1,0 from left to right.

Result: PASS. A mask represents every 4 x 4 grid exactly once in MSB-left order.

2. Construct the 16-bit mask

A set virtual pixel contributes its power of two. Multiple pixels are combined with bitwise OR. Clearing uses AND NOT; toggling uses XOR. The font never guesses a shape: the complete 16-bit mask is the shape.

example geometry mask 0x36C8

3	2	1	0
7	6	5	4
11	10	9	8
15	14	13	12

```
row 0: bit 3 = 0x0008
row 1: bits 6,7 = 0x0040 + 0x0080
row 2: bits 9,10 = 0x0200 + 0x0400
row 3: bits 12,13 = 0x1000 + 0x2000
OR total = 0x36C8

Binary: 0011 0110 1100 1000
```

GATE 2 - mask fidelity

Criterion to advance: Encoding and decoding must preserve every selected pixel.

Measured evidence: For every one of the geometry test's 2,197 cell/row/tick phase cases, the audit decoded all 16 mask bits and compared them with the independent analytic pixel predicate. Mismatches: 0.

Result: PASS. The 16-bit mask preserves the intended virtual pixels.

Boolean update examples

```
Set bit 10: old_mask OR 0x0400. Clear bit 10: old_mask AND NOT 0x0400. Toggle bit 10:
old_mask XOR 0x0400. Replace only after the updated 16-bit mask has been converted to its
codepoint.
```

3. Select Part 0 or Part 1 and calculate the codepoint

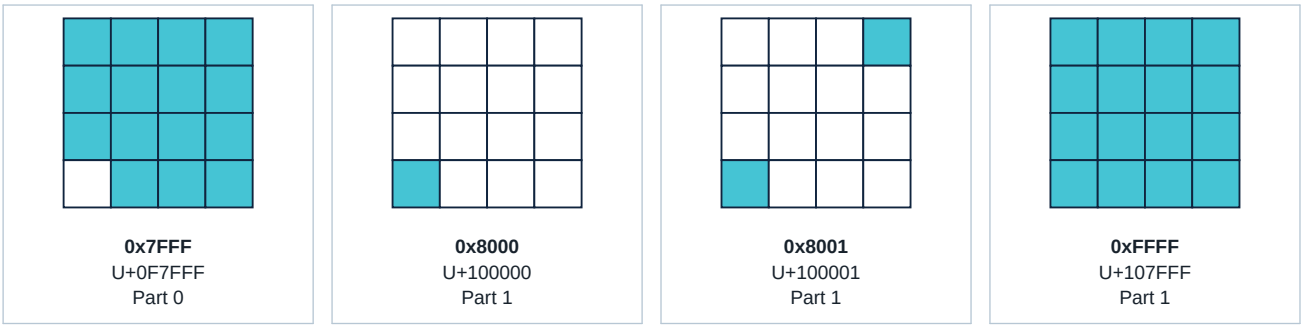
One font cannot place 65,536 consecutive glyphs into the remaining contiguous space of a single supplementary Private Use Area segment used here. The mask's most significant bit selects the font part; the remaining value gives the offset.

```
P0: if mask < 0x8000, codepoint = 0xF0000 + mask
P1: if mask >= 0x8000, codepoint = 0x100000 + (mask - 0x8000)
```

Example 0xC631

0xC631 has bit 15 set, so Part 1 is required. Offset = 0xC631 - 0x8000 = 0x4631. Codepoint = U+100000 + 0x4631 = **U+104631**.

The exact split boundary



0x7FFF
U+0F7FFF
Part 0

0x8000
U+100000
Part 1

0x8001
U+100001
Part 1

0xFFFF
U+107FFF
Part 1

Important: 0x7FFF followed numerically by 0x8000 is not a spatial animation. Binary rollover clears bits 0..14 and sets bit 15, so their pictures should be radically different. The split changes storage font, not the mathematical mask.

GATE 3 - codepoint uniqueness

Criterion to advance: Every mask must map to one valid, unique codepoint and reverse without loss.

Measured evidence: Independent enumeration produced 65,536 unique codepoints for 65,536 masks. Boundary results are P0 0x7FFF -> U+0F7FFF and P1 0x8000 -> U+100000.

Result: PASS. No collision, gap within a part, or off-by-one boundary error exists.

4. Verify the TrueType cmap and glyph index

The Unicode codepoint is useful only if the font's cmap points to the intended glyph. Supplementary-plane codepoints require cmap format 12. The audit reads the binary tables directly with FontTools; it does not rely on the demo's mapping function.

Part	Mask range	Codepoint range	Patterns	cmap 12	SHA-256
0	0000-7FFF	U+0F0000-U+0F7FFF	32,768	yes	b34587617903d811...
1	8000-FFFF	U+100000-U+107FFF	32,768	yes	ccfad9f530ceda3f...

Glyph-index rule

Each font begins with glyph IDs 0..17 (.notdef, space and sixteen pixel components). Therefore glyph ID = 18 + part_offset. For 0xC631 in Part 1, offset 0x4631 = 17,969 and glyph ID = 17,987. The binary cmap contains U+104631 -> glyph ID 17,987.

GATE 4 - cmap identity

Criterion to advance: For every codepoint, cmap must select the glyph at the offset dictated by its mask.

Measured evidence: Both binaries contain format 12 cmaps. All 32,768 entries in each part were checked. For every offset, glyph ID equalled 18+offset. Installed-file hashes exactly match the reproducible build hashes.

Result: PASS. The selected codepoint reaches the intended glyph in both parts.

5. Verify glyph components and physical outline bounds

Each pattern glyph is a TrueType composite of up to sixteen reusable rectangle components. A component may appear only when its corresponding mask bit is set. Every transform must be the identity transform.

Font metrics: 1000 units/em; advance 500; ascent 800; descent 200. In font v0.3, every local pixel is exactly 125 units wide by 250 units high. Edge overfill is zero: all component outlines remain within x=0..500 and y=-200..800.

bit	local	x bounds	y bounds	measured = expected
0	(3,0)	375..500	550..800	yes
1	(2,0)	250..375	550..800	yes
2	(1,0)	125..250	550..800	yes
3	(0,0)	0..125	550..800	yes
4	(3,1)	375..500	300..550	yes
5	(2,1)	250..375	300..550	yes
6	(1,1)	125..250	300..550	yes
7	(0,1)	0..125	300..550	yes
8	(3,2)	375..500	50..300	yes
9	(2,2)	250..375	50..300	yes
10	(1,2)	125..250	50..300	yes
11	(0,2)	0..125	50..300	yes
12	(3,3)	375..500	-200..50	yes
13	(2,3)	250..375	-200..50	yes
14	(1,3)	125..250	-200..50	yes
15	(0,3)	0..125	-200..50	yes

GATE 5 - outline identity and equal geometry

Criterion to advance: Every pattern glyph must contain exactly the components named by its mask, and all sixteen reusable components must be equal 125 x 250 rectangles wholly inside the cell.

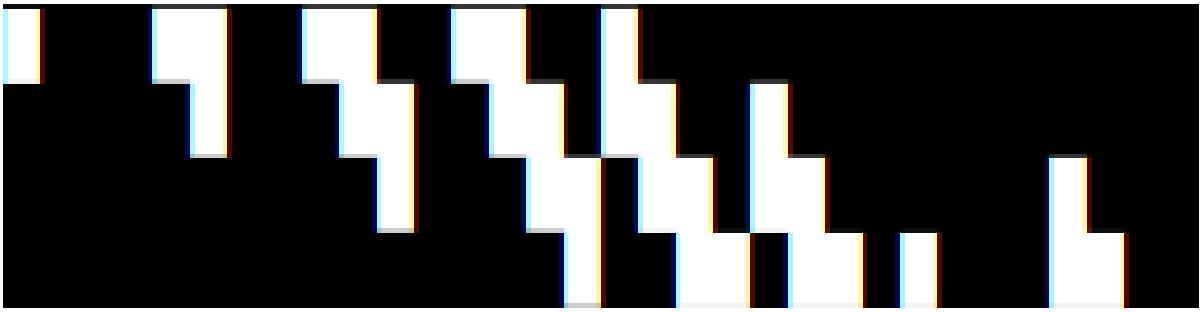
Measured evidence: All 65,536 v0.3 pattern glyphs were parsed. Component-ID sets exactly matched each mask's MSB-left physical position; transforms were identity. All sixteen component bounds matched the independent exact-grid calculation, and edge_overfill=0.

Result: PASS for v0.3. v0.2 is retained as the LSB-left legacy mapping.

6. Verify the installed Linux renderer

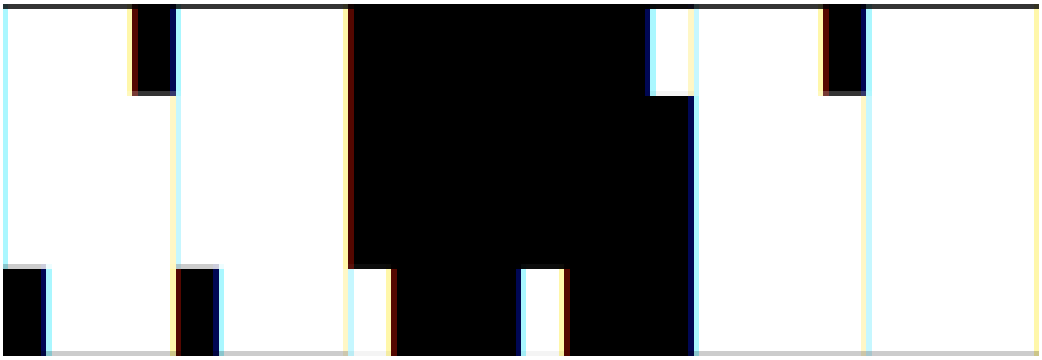
Structural correctness is not enough: the corrected v0.3 fonts must be selected and rasterized. The following strips were rendered remotely by pango-view through the installed Fontconfig alias, not drawn by this PDF.

Geometry-test glyph strip



Left to right: 0001, 0013, 0136, 136C, 36C8, 6C80, 8000, C800. These are the nine analytical geometry masks except blank 0000. Pango reported zero unknown glyphs and selected both PUA 4x4 parts.

Boundary glyph strip



Left to right: 7FFE, 7FFF, 8000, 8001, FFFE, FFFF. The abrupt visual change at 7FFF/8000 is the correct binary rollover described in Gate 3.

GATE 6 - runtime font selection

Criterion to advance: The installed stack must render both PUA ranges with their intended fonts and no missing-glyph fallback.

Measured evidence: Fontconfig selected PUA 4x4 Part 0 for U+0F0001 and Part 1 for U+100000. Pango serialized both family runs and reported unknown-glyphs=0. wcwidth returned one column for all boundary probes.

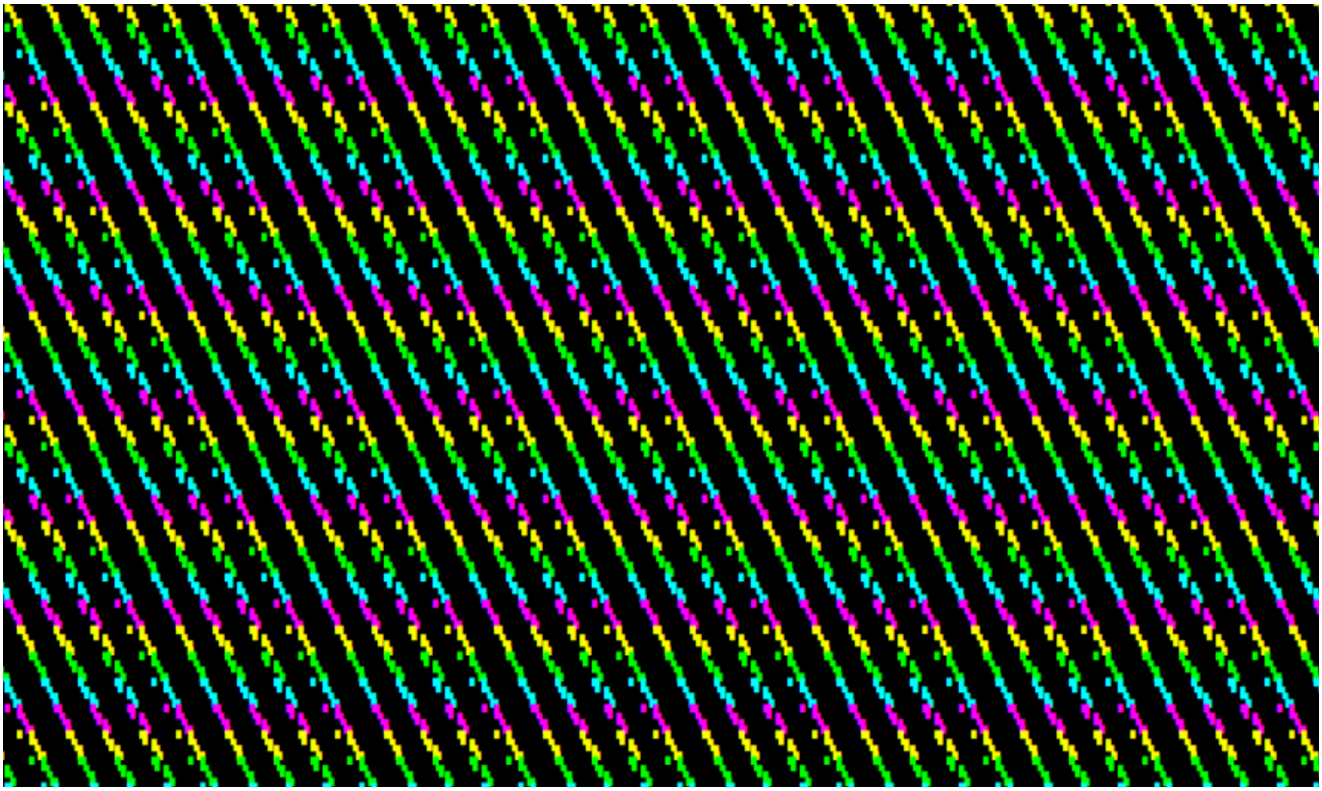
Result: PASS. Runtime selection is correct across the font boundary.

7. Explain the supplied geometry-test screenshots

The screenshots are consistent with the program's explicit geometry formula, not with random glyph substitution. The test sets a pixel when:

```
(virtual_x - virtual_y - tick) mod 13 is 0 OR 1
```

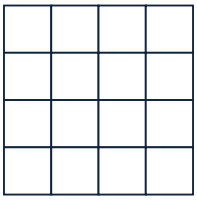
Selecting residues 0 and 1 deliberately makes each descending diagonal band two virtual pixels thick. Repeating modulo 13 creates blank intervals. When a band enters or leaves a 4 x 4 cell, the cell legitimately contains a small corner or hook fragment. The color is selected by terminal row and repeats cyan/yellow/green/magenta every four character rows; color does not encode the font part.



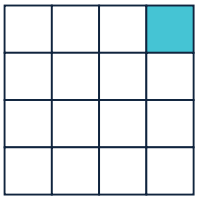
Supplied screenshot. The repeating two-pixel bands, blank periods and four-row palette cycle match the source predicate exactly.

8. Enumerate every shape the geometry test can generate

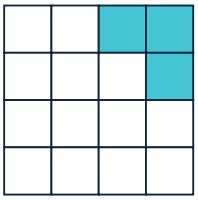
Across all relative cell positions and all thirteen animation phases, the test can generate only the following nine masks. Nothing else is emitted by the subpixels stage.



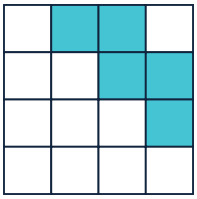
0x0000
U+0F0000
Part 0



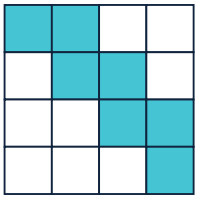
0x0001
U+0F0001
Part 0



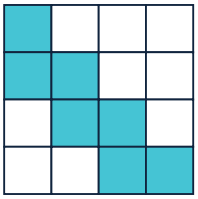
0x0013
U+0F0013
Part 0



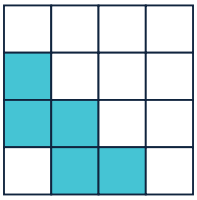
0x0136
U+0F0136
Part 0



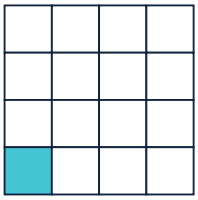
0x136C
U+0F136C
Part 0



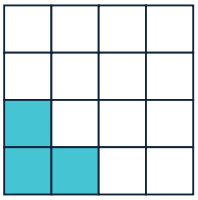
0x36C8
U+0F36C8
Part 0



0x6C80
U+0F6C80
Part 0



0x8000
U+100000
Part 1



0xC800
U+104800
Part 1

Only 0x8000 and 0xC800 use Part 1 because they set bit 15, the bottom-left local pixel. Their decoded shapes continue the same diagonal band as the Part 0 masks around them.

GATE 7 - geometry reconstruction

Criterion to advance: Decoding every emitted glyph must reproduce the analytic diagonal predicate in every local pixel.

Measured evidence: The independent audit tested 2,197 combinations of cell column, cell row and tick; each combination compared all sixteen decoded pixels. Decode mismatches: 0.

Result: PASS. The visible hook/corner fragments are expected partial bands, not wrong codepoints.

9. Seam evidence and final determination

A full-cell field exercises the maximum mask 0xFFFF, which is in Part 1 at U+107FFF. Any glyph-width, ascent, descent or exterior-bound error would appear as black seams between adjacent cells or rows.

Installed Pango seam matrix

Size	Raster	Black seam pixels	Result
8 px	256x129	0	PASS
9 px	320x145	0	PASS
10 px	320x160	0	PASS
11 px	384x177	0	PASS
12 px	384x193	0	PASS
13 px	448x209	0	PASS
14 px	448x225	0	PASS
16 px	512x257	0	PASS
18 px	576x289	0	PASS
20 px	640x320	0	PASS

GATE 8 - seamless solid mass

Criterion to advance: A solid 0xFFFF field must contain no black pixel between characters or rows at tested small raster sizes.

Measured evidence: Corrected v0.3 installed Pango rasters at 8,9,10,11,12,13,14,16,18 and 20 pixels contained zero black seam pixels even with edge_overfill=0.

Result: PASS. Overfill is not required for seamless solid rendering in the tested stack.

Final determination

Three observations are now separated. v0.1 had unequal exterior outlines. v0.2 fixed that geometry but encoded rows LSB-left. v0.3 preserves exact geometry and re-encodes every glyph MSB-left. Exhaustive mapping, component, trail-step, runtime-selection and seam tests now pass against the requested formula.

Appendix A - reproducible commands and artifacts

Independent full audit

```
cd experiments/pua-4x4
python3 audit_pua4x4_chain.py build --edge-overfill 0 --output
output/audit/pua4x4-v0.3-audit.json
```

Installed Linux audit

```
cd $HOME/dev/FontMaker/pua4x4
python3 audit_pua4x4_chain.py $HOME/.local/share/fonts --edge-overfill 0 --output
output/audit/installed-pua4x4-v0.3-audit.json
```

Existing exhaustive audit

```
python3 verify_pua4x4.py build
python3 verify_equal_pixel_geometry.py build
python3 verify_trail_steps.py
python3 verify_linux_runtime.py
python3 verify_pango_seams.py --font build --sizes 8,9,10,11,12,13,14,16,18,20
```

Evidence artifacts

pua4x4-v0.3-audit.json - corrected local reproducible-build audit
installed-pua4x4-v0.3-audit.json - corrected installed-binary audit
installed-v0.3-geometry-layout.json - Pango run selection and unknown-glyph count
installed-v0.3-boundary-layout.json - P0/P1 boundary Pango evidence
installed-seam-audit.txt - measured solid-raster seam matrix

Verified binary identities

```
v0.3 PUA4x4Part0.ttf
b34587617903d8115d8df788b6430b172c614d8fa9d1689eb403a5c8d26f8c6d

v0.3 PUA4x4Part1.ttf
ccfad9f530ceda3f33791aec877b81b81472604e68c5e1633c50bb6d2da2681a

Superseded v0.1 and v0.2 binaries are preserved under legacy/.
```

Audit status: PASS for v0.3. Mapping: bit=4*y+(3-x). Patterns verified: 65,536. Unique codepoints: 65,536. Equal components: 16/16 in both parts. Trail left-step regression: PASS. Geometry phase cases: 2,197. Geometry decode mismatches: 0. Pango unknown glyphs: 0.